

Framework Specialization Across Environments

Environment	Characteristics	Example Frameworks	Key Optimizations
Cloud	Abundant resources, scalability	TensorFlow, PyTorch	Distributed computing, large models
Edge	Moderate resources, low latency	TensorFlow Lite, Edge Impulse	Real-time inference, heterogeneous hardware
Mobile	Power constraints, varied hardware	TensorFlow Lite, Core ML	Energy efficiency, model compression
TinyML	Severe constraints, microcontrollers	TensorFlow Lite Micro	Extreme compression, static allocation *

* Static memory allocation reserves a fixed amount of memory during compile time, is faster to access, simpler to use, and is typically stored on the stack, though it requires memory needs to be known beforehand. Dynamic memory allocation, conversely, occurs during runtime, offers flexibility to change memory size, uses the heap, but requires careful management to prevent memory leaks and is generally slower.

Automated Compilation Pipelines - XLA

```
import tensorflow as tf

# Simple model component
class Block(tf.Module):
    def __init__(self, c): super().__init__(); self.conv =
tf.keras.layers.Conv2D(c, 3, padding="same")
    def __call__(self, x, training=False):
        x = self.conv(x)
        x = tf.keras.layers.BatchNormalization()(x,
training=training)
        return tf.nn.relu(x)

# Enable XLA for this function (TF >= 2.11: jit_compile=True)
@tf.function(jit_compile=True)
def step(x):
    b = Block(32)
    return b(x, training=False)

# Warm up + run
x = tf.random.normal([1, 224, 224, 3])
y = step(x) # XLA compiles on first call; later calls use
optimized code
```

What XLA (Accelerated Linear Algebra) does

- Fuses ops into single kernels
- Picks efficient layouts (e.g., NHWC $\leftrightarrow$ NCHW)
- Reuses buffers to cut memory

When to use

- TensorFlow models with stable shapes
- Training or inference on CPU/GPU/TPU

When to be careful

- Highly dynamic Python control flow
- Ragged tensors or frequent shape changes

How to enable

- `@tf.function(jit_compile=True)` in TF 2.11+ (or `experimental_compile=True` in older TF)
- Or set `TF_XLA_FLAGS / tf.config.optimizer.set_jit(True)`

Automated Compilation Pipelines - TVM

What TVM does

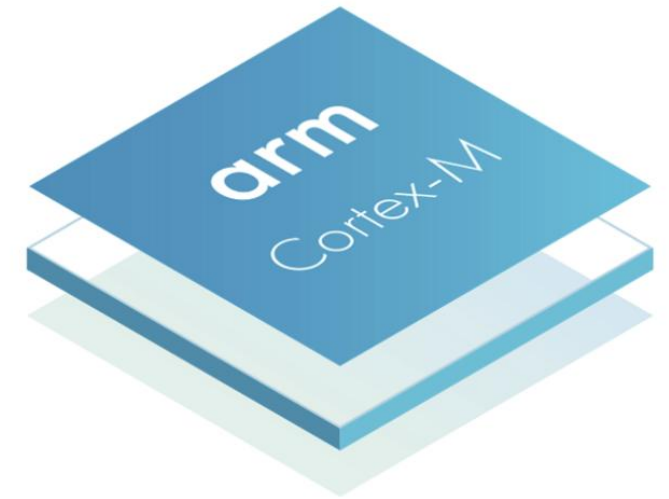
- Cross-framework import → auto-schedule/auto-tune → target-specific codegen
- Generates kernels for CPUs, GPUs, and MCUs

Why it's different from XLA

- Hardware-agnostic stack with search-based optimization
- Produces standalone artifacts (C/C++/LLVM) for deployment

When to use

- Heterogeneous fleets or non-TF models
- Squeezing perf on edge devices (ARM NEON, DSP, Tensor Cores)



ARM Cortex-M processors are a family of 32-bit RISC (Reduced Instruction Set Computer) microcontroller cores designed by Arm Limited. They are widely used in embedded systems, particularly for low-cost and energy-efficient applications.

Image source: community.arm.com

```
import torch, torch.nn as nn, tvm
from tvm import relay

m = nn.Sequential(nn.Conv2d(3,8,3,padding=1), nn.ReLU(), nn.AvgPool2d(2)).eval()
ex = torch.randn(1,3,64,64); ts = torch.jit.trace(m, ex).eval()
mod, params = relay.frontend.from_pytorch(ts, [("input0", ex.shape)])

target = tvm.target.Target("c -mcpu=cortex-m4") # emit portable C for Cortex-M4
with tvm.transform.PassContext(opt_level=3):
    lib = relay.build(mod, target=target, params=params)

open("model.c", "w").write(lib.get_source()) # compile with your MCU toolchain
```

Automated Compilation Pipelines - MLIR

3. Multi-Level Intermediate Representation (MLIR):

MLIR is **not a compiler itself** but a framework for building compilers. It enables **multi-stage optimizations** across different abstraction levels.

Key Features:

- **Flexible IR:** Can represent high-level (e.g., TensorFlow ops) and low-level (e.g., LLVM IR) code.
- **Domain-Specific Optimizations:** Supports dialects (e.g., linalg for linear algebra, gpu for GPU ops).
- **Reusable Passes:** Compiler transformations can be shared across frameworks.

How It Works:

A model is progressively lowered from high-level ops to hardware-specific instructions.

Example flow:

TensorFlow Graph → **MLIR (tf dialect)** → **MLIR (linalg dialect)** → **LLVM IR** → **Machine Code**

- `tf dialect` = TensorFlow-specific ops.
- `linalg dialect` = Generalized linear algebra ops for compiler-friendly optimizations.

Use Case:

Used internally in TensorFlow, PyTorch, and TVM for **flexible compiler pipelines**.